SECURITY AUDIT REPORT

OWASP Juice Shop

github.com/juice-shop/juice-shop | commit 717f5efa8 | audited 30 Oct 2026

Read this first. Juice Shop is a deliberately insecure training application, so most findings below are *intentional*. This report shows what a real audit of an unfamiliar repository produces, and how each class of issue would be fixed in a production app. Do not deploy Juice Shop on a public server.

1. Executive summary

Category	Critical	High	Medium	Low
Dependencies npm audit, 40 vulnerable packages	7	21	10	2
Code and secrets manually verified in source	3	2	4	0

Overall risk: critical. The application can be taken over without credentials (SQL injection in login), its token-signing private key is published in the source, and its dependency tree contains critical flaws in libraries that handle authentication (`jsonwebtoken`, `express-jwt`), archives and templating. Three direct dependencies (`marsdb`, `hbs`, `notevil`) have no fixed version and must be replaced.

Top priorities

1	Replace string-built SQL with parameterized queries (C1, C2)	Removes remote database takeover and login bypass
2	Remove the private key from source, rotate it, purge git history (C3)	Anyone can currently forge valid login tokens
3	Upgrade jsonwebtoken, express-jwt, sanitize-html; replace marsdb, hbs, notevil	Closes critical/high auth, XSS and injection flaws in dependencies
4	Replace MD5 password hashing with argon2id or bcrypt (H1)	Stolen password hashes become impractical to crack
5	Move secrets to environment or a secret manager (H2) and tighten CORS / CSP (M2, M3)	Limits blast radius and browser-side attacks

2. Scope, method and limits

The audit used `audit_repo.sh` (steps below) and then manual source review of the locations the scanners flagged. Every finding in section 3 was confirmed by reading the code; section 4 comes directly from `npm audit`.

Check	Tool	Status	Notes
Repo overview and churn hotspots	git	Done	1,468 tracked files, 180 contributors; package.json and server.js are the most-changed files
Dependency vulnerabilities	npm audit	Done	40 findings (section 4). A temporary lockfile was generated because the repo gitignores its lockfile
Secrets in files and git history	Gitleaks	Done	212 raw hits, 27 outside test paths. Reviewed; see section 5
Dependency scan (second opinion)	OSV-Scanner	**Not completed**	Read 1,181 packages but its vulnerability API was unreachable from the audit environment
Dependencies, misconfig, secrets	Trivy	**Not completed**	Vulnerability database download blocked by the environment
Static analysis rules	Semgrep	**Not completed**	Rule download blocked. Source review in section 3 was manual instead
Build and test run	npm test	Not run	Opt-in because it executes repo code; run inside Docker or a VM
Dynamic testing of the running app	none	Not run	No exploitation was attempted; findings are from source and dependency data

Because three scanners did not complete, the absence of a finding is not evidence of safety. The counts in this report are a lower bound. Re-run Trivy, Semgrep and OSV-Scanner on a machine with internet access and merge the results.

How severity is assigned

Critical: remotely exploitable, no or little authentication, full compromise of data or accounts. **High**: serious weakness that needs a further condition (e.g. stolen data, a second flaw). **Medium**: hardening gaps and attacks that need user interaction. **Low**: minor or hard to exploit. Dependency severities are those published in the npm advisory database; code severities are the auditor's judgement.

What was done well

The container build is sound: a multi-stage Dockerfile that ships a `distroless` runtime image, runs as a non-root user (UID 65532), and removes dev artefacts. The X-Powered-By header is disabled, and `noSniff` and `frameguard` headers are set.

3. Code and configuration findings

CRITICAL C1 SQL injection in login

Location	`routes/login.ts:34`
What is wrong	The login query is built by concatenating `req.body.email` directly into the SQL string.
Impact	An attacker can log in as any user, including the administrator, without a password, or extract the database.
How to fix	Use bound parameters and never interpolate user input. Hash comparison should happen in code with a slow password hash (see H1).

routes/login.ts

```
// BEFORE (vulnerable)
`SELECT * FROM Users WHERE email = '${req.body.email}' AND password = '...'`

// AFTER
const user = await UserModel.findOne({ where: { email: req.body.email, deletedAt: null } })
if (!user || !(await argon2.verify(user.password, req.body.password))) return res.status(401).end()
```

CRITICAL C2 SQL injection in product search

Location	`routes/search.ts:23`
What is wrong	The search term `req.query.q` is placed inside a `LIKE '%...%'` clause in a raw query.
Impact	UNION-based injection lets an attacker read any table, including users and their password hashes.
How to fix	Use named replacements so the database treats the value as data, not SQL.

routes/search.ts

```
models.sequelize.query(
  "SELECT * FROM Products WHERE (name LIKE :q OR description LIKE :q) AND deletedAt IS NULL ORDER BY name",
  { replacements: { q: `%${criteria}%` } }
)
```

CRITICAL C3 Private signing key committed to source (and reused)

Location	`lib/insecurity.ts:21 and infrastructure/terraform/networking.tf:171`
What is wrong	An RSA private key is hard-coded as a string. The same key appears in the Terraform file and is also used as an HMAC secret (line 150).
Impact	Anyone who can read the repository can mint valid JWTs for any user or role (administrator included). Deleting the line does not help: the key remains in git history.
How to fix	Treat the key as compromised. Generate a new key pair, load it from an environment variable or secret manager, use separate keys for signing and HMAC, and remove the old key from history with `git filter-repo` or BFG. Rotate anything that was signed with it.

lib/insecurity.ts

```
// AFTER
const privateKey = process.env.JWT_PRIVATE_KEY   // injected at deploy time
if (!privateKey) throw new Error('JWT_PRIVATE_KEY is not set')
export const authorize = (user = {}) =>
  jwt.sign(user, privateKey, { expiresIn: '1h', algorithm: 'RS256' })
```

HIGH H1 Passwords hashed with unsalted MD5

Location	`lib/insecurity.ts:41`
What is wrong	`crypto.createHash('md5')` is used for passwords. MD5 is fast and has no salt.
Impact	If the database leaks (see C1/C2), most passwords can be cracked in seconds with precomputed tables.
How to fix	Use argon2id (preferred) or bcrypt with a per-user salt. Migrate existing users by re-hashing at their next successful login.

lib/insecurity.ts

```
import argon2 from 'argon2'
export const hash = (pw: string) => argon2.hash(pw, { type: argon2.argon2id })
export const verifyPw = (h: string, pw: string) => argon2.verify(h, pw)
```

HIGH H2 Hard-coded HMAC secret

Location	`lib/insecurity.ts:42`
What is wrong	A literal string is used as the HMAC key in source code.
Impact	Anyone with the code can forge values that the application trusts as authentic.
How to fix	Load from an environment variable or secret manager, rotate it, and keep different secrets per environment.

lib/insecurity.ts

```
const hmacKey = process.env.HMAC_SECRET!
export const hmac = (d: string) => crypto.createHmac('sha256', hmacKey).update(d).digest('hex')
```

MEDIUM M1 Redirect allow-list can be bypassed

Location	`lib/insecurity.ts:136`
What is wrong	The allow-list test uses `url.includes(allowedUrl)`, so any URL that merely contains an allowed string passes (for example `https://evil.com/?https://github.com/...`).
Impact	Open redirect, useful for phishing and for stealing OAuth codes.
How to fix	Parse the URL and compare the exact origin (and path if needed) against the list.

lib/insecurity.ts

```
const ok = (u: string) => { try { return ALLOWED_ORIGINS.has(new URL(u).origin) } catch { return false } }
```

MEDIUM M2 CORS allows every origin

Location	`server.ts:183-184`
What is wrong	`app.use(cors())` with no options reflects any origin.
Impact	Any website a user visits can call the API from their browser. Combined with token-based auth this widens the attack surface.
How to fix	Restrict to the origins you control.

server.ts

```
app.use(cors({ origin: ['https://shop.example.com'], credentials: true }))
```

MEDIUM M3 Security headers incomplete (no CSP)

Location	`server.ts:187-189`
What is wrong	Only `noSniff` and `frameguard` are enabled. No Content-Security-Policy or HSTS was found in the code that was reviewed.
Impact	Without a CSP, any XSS (including via the vulnerable sanitize-html versions in section 4) runs unrestricted.
How to fix	Enable helmet's defaults and define a strict CSP.

server.ts

```
app.use(helmet({ contentSecurityPolicy: { directives: { defaultSrc: ["'self'"] } } }))
```

MEDIUM M4 JWT verification does not pin the algorithm (needs confirmation)

Location	`lib/insecurity.ts:52`
What is wrong	`expressJwt({ secret: publicKey } as any)` uses a type cast that hides the library's requirement to declare accepted algorithms.
Impact	If algorithms are not restricted, algorithm-confusion attacks on tokens become possible. Combined with the known flaws in the pinned express-jwt/jsonwebtoken versions (section 4) this is significant. *This item was identified from code reading and should be confirmed by testing.*
How to fix	Upgrade express-jwt, remove the cast, and set the algorithm explicitly.

lib/insecurity.ts

```
expressJwt({ secret: publicKey, algorithms: ['RS256'] })
```

4. Vulnerable dependencies (npm audit)

40 packages are affected: 7 critical, 21 high, 10 medium, 2 low. *Direct* means the project depends on it itself; the others are pulled in by a parent and are fixed by upgrading that parent. Most fixes are major-version upgrades and need a test run. Where npm's suggested fix is really a downgrade (every newer release is affected), the Fix column says so. Juice Shop pins some old versions on purpose.

Critical (7)

Package	Direct	Affected	Issue	Fix
`jsonwebtoken`	Yes	<=8.5.1	Verification Bypass in jsonwebtoken (+3 more advisories)	Upgrade **jsonwebtoken** to 9.0.3 (major, test!)
`marsdb`	Yes	*	Command Injection in marsdb	**No fix.** Replace or remove
`pdfkit`	Yes	0.9.0 - 0.12.1	Inherited from: crypto-js	Upgrade **pdfkit** to 0.20.2 (major, test!)
`crypto-js`	No	<=4.1.1	crypto-js PBKDF2 1,000 times weaker than specified in 1993 and 1.3M times weaker than current standard (+1 more advisories)	Upgrade **pdfkit** to 0.20.2 (major, test!)
`decompress`	No	*	Decompress: Archive extraction can create files and links outside of the target directory (+3 more advisories)	Use an npm **override** to pin a patched **decompress**. npm's suggested route (download@4.4.3) does not truly resolve it
`lodash`	No	<=4.17.23	Prototype Pollution in lodash (+4 more advisories)	Upgrade **sanitize-html** to 2.18.1 (major, test!)
`tar`	No	<=7.5.20	node-tar Vulnerable to Arbitrary File Creation/Overwrite via Hardlink Path Traversal (+11 more advisories)	Upgrade **sqlite3** to 6.0.1 (major, test!)

High (21)

Package	Direct	Affected	Issue	Fix
`@cyclonedx/cyclonedx-npm`	Yes	<=5.0.0	@cyclonedx/cyclonedx-npm: Shell Injection via Unsanitized --workspace Argument (+1 more advisories)	Upgrade **@cyclonedx/cyclonedx-npm** to 2.0.0
`express-jwt`	Yes	<=7.7.7	Authorization bypass in express-jwt	Upgrade **express-jwt** to 8.5.1 (major, test!)
`proxy-agent`	Yes	>=6.0.0	Inherited from: pac-proxy-agent	**No patched release.** npm suggests proxy-agent@5.0.0, which is older or still affected. Replace the package or pin and review
`replace`	Yes	>=0.2.3	Inherited from: minimatch	**No patched release.** npm suggests replace@0.2.2, which is older or still affected. Replace the package or pin and review
`sanitize-html`	Yes	<=2.17.7	Improper Input Validation in sanitize-html (+9 more advisories)	Upgrade **sanitize-html** to 2.18.1 (major, test!)
`sqlite3`	Yes	5.0.0 - 5.1.7	Inherited from: node-gyp, tar	Upgrade **sqlite3** to 6.0.1 (major, test!)

Package	Direct	Affected	Issue	Fix
`basic-ftp`	No	<=6.2.0	basic-ftp: Quadratic-time CPU denial of service in Client.list() Unix directory-listing parser (RE_LINE backtracking)	Use an npm **override** to pin a patched **basic-ftp**. npm's suggested route (proxy-agent@5.0.0) does not truly resolve it
`cacache`	No	14.0.0 - 18.0.4	Inherited from: tar	Upgrade **sqlite3** to 6.0.1 (major, test!)
`cacheable-request`	No	0.1.0 - 2.1.4	Inherited from: http-cache-semantics, normalize-url	Use an npm **override** to pin a patched **cacheable-request**. npm's suggested route (download@4.4.3) does not truly resolve it
`decode-uri-component`	No	<=0.5.0	decode-uri-component: Denial of service via exponential decoding of malformed percent-encoded input (+1 more advisories)	Use an npm **override** to pin a patched **decode-uri-component**. npm's suggested route (download@4.4.3) does not truly resolve it
`get-uri`	No	>=5.0.0	Inherited from: basic-ftp	**No patched release.** npm suggests proxy-agent@5.0.0, which is older or still affected. Replace the package or pin and review
`got`	No	<=11.8.3	Got allows a redirect to a UNIX socket	Use an npm **override** to pin a patched **got**. npm's suggested route (download@4.4.3) does not truly resolve it
`http-cache-semantics`	No	<4.1.1	http-cache-semantics vulnerable to Regular Expression Denial of Service	Use an npm **override** to pin a patched **http-cache-semantics**. npm's suggested route (download@4.4.3) does not truly resolve it
`jws`	No	<=3.2.2	Forgeable Public/Private Tokens in jws (+1 more advisories)	Upgrade **jsonwebtoken** to 9.0.3 (major, test!)
`make-fetch-happen`	No	7.1.1 - 14.0.0	Inherited from: cacache, http-proxy-agent	Upgrade **sqlite3** to 6.0.1 (major, test!)
`minimatch`	No	<=3.1.3	minimatch has a ReDoS via repeated wildcards with non-matching literal in pattern (+2 more advisories)	Use an npm **override** to pin a patched **minimatch**. npm's suggested route (replace@0.2.2) does not truly resolve it
`moment`	No	<=2.29.1	Regular Expression Denial of Service in moment (+2 more advisories)	Upgrade **jsonwebtoken** to 9.0.3 (major, test!)
`node-gyp`	No	<=10.3.1	Inherited from: make-fetch-happen, tar	Upgrade **sqlite3** to 6.0.1 (major, test!)
`normalize-url`	No	2.0.0 - 2.0.1	Inherited from: query-string	Use an npm **override** to pin a patched **normalize-url**. npm's suggested route (download@4.4.3) does not truly resolve it

Package	Direct	Affected	Issue	Fix
`pac-proxy-agent`	No	>=6.0.0	Inherited from: get-uri	**No patched release.** npm suggests proxy-agent@5.0.0, which is older or still affected. Replace the package or pin and review
`query-string`	No	5.0.0 - 9.4.1	Inherited from: decode-uri-component	Use an npm **override** to pin a patched **query-string**. npm's suggested route (download@4.4.3) does not truly resolve it

Medium (10)

Package	Direct	Affected	Issue	Fix
`download`	Yes	<=2.0.0 || >=4.0.0	Inherited from: decompress, got	**No patched release.** npm suggests download@4.4.3, which is older or still affected. Replace the package or pin and review
`file-type`	Yes	13.0.0 - 21.3.0	file-type affected by infinite loop in ASF parser on malformed input with zero-size sub-header	Upgrade **file-type** to 22.3.0 (major, test!)
`hbs`	Yes	*	hbs: Insertion of Sensitive Information into Externally-Accessible File or Directory and Exposure of Sensitive Information to an Unauthorized Actor	**No fix.** Replace or remove
`notevil`	Yes	*	Sandbox escape in notevil and argencoders-notevil	**No fix.** Replace or remove
`nyc`	Yes	15.0.0-alpha.0 - 17.1.0	Inherited from: istanbul-lib-processinfo	Upgrade **nyc** to 18.0.0 (major, test!)
`sequelize`	Yes	0.0.0-development || >=3.30.1	Inherited from: uuid	**No patched release.** npm suggests sequelize@3.30.0, which is older or still affected. Replace the package or pin and review
`base64url`	No	<3.0.0	Out-of-bounds Read in base64url	Upgrade **jsonwebtoken** to 9.0.3 (major, test!)
`istanbul-lib-processinfo`	No	<=3.0.0	Inherited from: uuid	Upgrade **nyc** to 18.0.0 (major, test!)
`jwa`	No	<=1.1.5	Inherited from: base64url	Upgrade **jsonwebtoken** to 9.0.3 (major, test!)
`uuid`	No	<11.1.1	uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided	Use an npm **override** to pin a patched **uuid**. npm's suggested route (sequelize@3.30.0) does not truly resolve it

Low (2)

Package	Direct	Affected	Issue	Fix
`@tootallnate/once`	No	<2.0.1	@tootallnate/once vulnerable to Incorrect Control Flow Scoping	Upgrade **sqlite3** to 6.0.1 (major, test!)

Package	Direct	Affected	Issue	Fix
`http-proxy-agent`	No	4.0.1	Inherited from: @tootallnate/once	Upgrade **sqlite3** to 6.0.1 (major, test!)

How to fix dependency findings

1. Run the safe fixes first, then review what remains:

```
npm audit fix            # non-breaking upgrades only
npm audit                # see what is left
```

2. Handle the breaking upgrades one at a time and run the tests after each: `npm install jsonwebtoken@9 express-jwt@8 sanitize-html@latest sqlite3@6`. Upgrading a parent usually clears its transitive entries (crypto-js, tar, lodash, jws, moment).
3. For packages with **no fix** (`marsdb`, `hbs`, `notevil`): remove them or switch to a maintained alternative; they will not be patched. `notevil` is a sandbox with a known escape and must not run untrusted input.
4. Commit the lockfile (this repo does not), pin versions, and enable Dependabot or Renovate plus `npm audit` in CI so new advisories fail the build.

5. Secrets scan

Gitleaks reported **212** raw matches (151 generic API keys, 54 JWT-shaped strings, 7 private keys), of which **164** are in `test/` and **27** are outside test, docs and fixture paths. Reviewed:

Location	Assessment
lib/insecurity.ts, infrastructure/terraform/networking.tf	**Confirmed critical**: real RSA private key in source (C3)
data/static/users.yml, data/data/users.json	Seeded demo accounts (24 password fields). Intentional for a training app. In a real project these would be critical
routes/login.js and login.ts (history)	Hard-coded credential strings in earlier commits; still present in git history
frontend/.../faucet.component.ts	Long string flagged as generic key; likely test data. Not individually verified

If a real secret is ever found: revoke and rotate it first (assume it is compromised), then remove it from the code, then purge history. Add `gitleaks` as a pre-commit hook and CI step to stop recurrence.

6. Remediation roadmap

When	Action	Findings	Effort
Immediately	Rotate and remove the private key and HMAC secret; purge history	C3, H2	Hours
Immediately	Parameterize the login and search queries	C1, C2	Hours
This week	Upgrade jsonwebtoken, express-jwt; pin JWT algorithms; add auth tests	M4, dependency critical/high	1 to 2 days
This week	Replace marsdb, hbs, notevil (no fixes exist)	Dependency critical/medium	1 to 3 days
Next sprint	Move to argon2id with migrate-on-login	H1	1 to 2 days
Next sprint	Fix redirect check, restrict CORS, add CSP and HSTS	M1, M2, M3	1 day
Ongoing	Lockfile committed; Dependabot; Gitleaks, Semgrep, Trivy in CI	All	1 day setup

7. Reproduce this audit

```shell

pip install semgrep   # plus trivy, gitleaks, osv-scanner
./audit_repo.sh https://github.com/juice-shop/juice-shop ./juice-report
# then verify the flagged lines by reading the source (as done for section 3)
# optional, inside Docker/VM only:  RUN_BUILD=1 ./audit_repo.sh <repo>
```

Next verification steps: start the app (`docker run -p 3000:3000 bkimminich/juice-shop`), confirm C1, C2 and M4 dynamically, and compare this report with the in-app score board to see which intentional flaws the scanners missed.